

Bloc 2 MVC PHP TD8

Contexte	2
Exercice 1	2
Exercice 2	5
Exercice 3	7
Exercice 4	8
Exercice 5	10
Exercice 6	12
Exercice 7	12
Exercice 8	14
Exercice 9	15
Exercice 10	16
Exercice 11	18
Exercice 12	19
Exercice 13	20
Exercice 14	22
Exercice 15	27
Conclusion	30

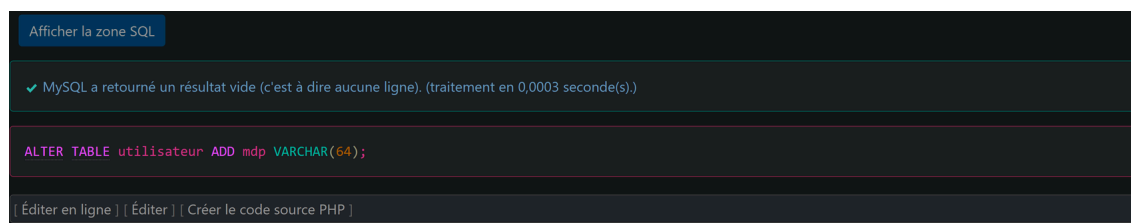
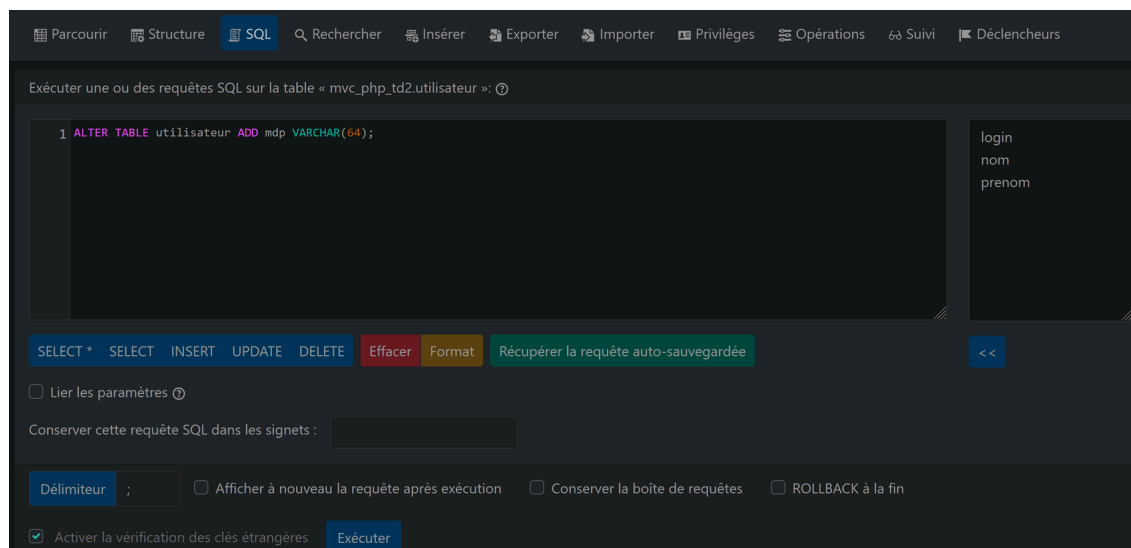
Contexte

Ce **TD** vient à la suite du **TD7** sur les cookies et les sessions. On met en place l'authentification par mot de passe des utilisateurs, on protège certaines actions (un utilisateur ne peut modifier que ses données, l'administrateur a tous les droits) et on valide l'inscription par email.

(Les fichiers projet seront disponibles dans le portfolio sur GitHub).

Exercice 1

Ajout d'une colonne **mdp** de type **VARCHAR(64)** dans la table **utilisateur** (64 caractères car on stockera le mot de passe chiffré en **SHA-256**) :



Ajout de deux champs `<input type="password">` dans le formulaire `"update.php"`, le deuxième sert à confirmer le premier :

```
<p>
  <label for="prenom_id">Prénom</label> :
  <input type="text" placeholder="Ex : Jean" name="prenom" id="prenom_id" value="<?php echo $uPrenom; ?>" required/>
</p>
<p>
  <label for="mdp_id">Mot de passe</label> :
  <input type="password" name="mdp" id="mdp_id" required/>
</p>
<p>
  <label for="mdp2_id">Confirmation du mot de passe</label> :
  <input type="password" name="mdp2" id="mdp2_id" required/>
</p>
<p>
  <input type="submit" value="Envoyer" />
</p>
</fieldset>
</form>
```

Dans les actions **created** et **updated**, on vérifie que les deux mots de passe sont identiques avant de sauver :

```
public static function created() {
    if ($_GET['mdp'] != $_GET['mdp2']) {
        self::error("les deux mots de passe ne correspondent pas.");
    } else {
        $data = array(
            "login" => $_GET['login'],
            "nom" => $_GET['nom'],
            "prenom" => $_GET['prenom'],
            "mdp" => $_GET['mdp'],
        );
        if (ModelUtilisateur::save($data) === false) {
            self::error("ce login est déjà utilisé.");
        } else {
            $tab_u = ModelUtilisateur::selectAll();
            $view = 'created';
            $pagetitle = 'Utilisateur créé';
            require File::build_path(array("view", "view.php"));
        }
    }
}
```

Voitures | [Utilisateurs](#) | [Trajets](#) | [Panier](#) | [Préférences](#)

Mon formulaire :

Login :

Nom :

Prénom :

Mot de passe :

Confirmation du mot de passe :

Voitures | [Utilisateurs](#) | [Trajets](#) | [Panier](#) | [Préférences](#)

L'utilisateur a bien été créé !

Liste des utilisateurs :

Utilisateur [jdupont](#) ([supprimer](#))

Utilisateur [mbardet](#) ([supprimer](#))

Utilisateur [pmartin](#) ([supprimer](#))

Utilisateur [raubree](#) ([supprimer](#))

[Ajouter un utilisateur](#)

Si les deux mots de passe sont différents :

Voitures | [Utilisateurs](#) | [Trajets](#) | [Panier](#) | [Préférences](#)

Erreur : les deux mots de passe ne correspondent pas.

[Retour à la liste](#)

Le mot de passe est bien enregistré... mais en clair dans la **BDD** :

Options supplémentaires

				login	nom	prenom	mdp
☐	Éditer	Copier	Supprimer	jdupont	Dupont	Jean	NULL
☐	Éditer	Copier	Supprimer	mbardet	Bardet	Marion	NULL
☐	Éditer	Copier	Supprimer	pmartin	Martin	Paul	NULL
☐	Éditer	Copier	Supprimer	raubree	Aubrée	Romain	romain05

Exercice 2

Création de la classe **Security** dans “lib/Security.php” avec la fonction **chiffrer** (hachage **SHA-256**) :

```
<?php
class Security {

    public static function chiffrer($texte_en_clair) {
        $texte_chiffre = hash('sha256', $texte_en_clair);
        return $texte_chiffre;
    }
}
?>
```

On fait un **require_once** de “**Security.php**” dans le contrôleur et on sauve le mot de passe chiffré :

```
public static function created() {  
    if ($_GET['mdp'] != $_GET['mdp2']) {  
        self::error("les deux mots de passe ne correspondent pas.")  
    } else {  
        $data = array(  
            "login" => $_GET['login'],  
            "nom" => $_GET['nom'],  
            "prenom" => $_GET['prenom'],  
            "mdp" => Security::chiffrer($_GET['mdp']),  
        );  
        if (ModelUtilisateur::save($data) === false) {  
            self::error("ce login est déjà utilisé.");  
        } else {  
            $tab_u = ModelUtilisateur::selectAll();  
            $view = 'created';  
            $pagetitle = 'Utilisateur créé';  
            require File::build_path(array("view", "view.php"));  
        }  
    }  
}
```

On remet le mot de passe de “**raubree**” avec l’action **update** :

Voitures | Utilisateurs | Trajets | Panier | Préférences

Mon formulaire :

Login :

Nom :

Prénom :

Mot de passe :

Confirmation du mot de passe :

Le mot de passe est maintenant haché :

Options supplémentaires

	login	nom	prenom	mdp
☐ Éditer ☐ Copier ☐ Supprimer	jdupont	Dupont	Jean	NULL
☐ Éditer ☐ Copier ☐ Supprimer	mbardet	Bardet	Marion	NULL
☐ Éditer ☐ Copier ☐ Supprimer	pmartin	Martin	Paul	NULL
☐ Éditer ☐ Copier ☐ Supprimer	raubree	Aubrée	Romain	7ef67ef1442475917d391cdc2a231286de23b7d955e7ea0661...

Exercice 3

1. On crée un utilisateur **“bidon”** avec un mot de passe très courant : **“password”** :

Voitures | Utilisateurs | Trajets | Panier | Préférences

Mon formulaire :

Login :

Nom :

Prénom :

Mot de passe :

Confirmation du mot de passe :

2. On regarde son mot de passe chiffré dans la **BDD** :

Options supplémentaires

	login	nom	prenom	mdp
☐ Éditer ☐ Copier ☐ Supprimer	bidon	Bidon	Test	5e884898da28047151d0e56f8dc6292773603d0d6aabdd62a...
☐ Éditer ☐ Copier ☐ Supprimer	jdupont	Dupont	Jean	NULL
☐ Éditer ☐ Copier ☐ Supprimer	mbardet	Bardet	Marion	NULL
☐ Éditer ☐ Copier ☐ Supprimer	pmartin	Martin	Paul	NULL
☐ Éditer ☐ Copier ☐ Supprimer	raubree	Aubrée	Romain	7ef67ef1442475917d391cdc2a231286de23b7d955e7ea0661...

3. Avec le site **md5decrypt**, on retrouve le mot de passe d'origine en quelques secondes :



Ce site stocke les hachés de milliards de mots de passe courants (attaque par dictionnaire), donc un mot de passe classique n'est pas protégé par le hachage seul.

Exercice 4

On génère une chaîne aléatoire sur **random.org** :



On l'utilise comme graine (**\$seed**) dans la classe **Security**, et **chiffrer** la concatène au début du mot de passe avant de le hacher (on dit qu'on "sale" le mot de passe) :

```
<?php
class Security {

    private static $seed = 'WQfUsVV9vQnDoR2a';

    static public function getSeed() {
        return self::$seed;
    }

    public static function chiffrer($texte_en_clair) {
        $texte_chiffre = hash('sha256', self::$seed . $texte_en_clair);
        return $texte_chiffre;
    }
}
```

On refait le test avec un nouvel utilisateur **"bidon2"** qui a aussi le mot de passe **"password"** :

	login	nom	prenom	mdp
☐ Éditer ☐ Copier ☐ Supprimer	bidon	Bidon	Test	5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a...
☐ Éditer ☐ Copier ☐ Supprimer	bidon2	Bidon	Deux	449e014b6fecfcd69d2cdd03fb57703cbb4c46f42efbcc2d8c...
☐ Éditer ☐ Copier ☐ Supprimer	jdupont	Dupont	Jean	NULL
☐ Éditer ☐ Copier ☐ Supprimer	mbardet	Bardet	Marion	NULL
☐ Éditer ☐ Copier ☐ Supprimer	pmartin	Martin	Paul	NULL
☐ Éditer ☐ Copier ☐ Supprimer	raubree	Aubrée	Romain	17e873217a7d38c155c6b7fb8acd2392881e1524ce292932c6...

Le haché est différent de celui de **"bidon"** et **md5decrypt** ne le trouve plus :

md5decrypt.net/Sha256/

SHA-256 : générateur et recherche de hash

ENTRÉE

449e014b6fecfcd69d2cdd03fb57703cbb4c46f42efbcc2d8caf823669c46fec

Générer le hash

RÉSULTAT

SHA256

Aucun des hashes soumis n'a été trouvé dans la base.

Rechercher dans la base

Exercice 5

1. Vue “**connect.php**” avec un formulaire login / mot de passe qui appelle l’action **connected** :

```
<form method="get" action="index.php">
  <fieldset>
    <legend>Connexion :</legend>
    <input type="hidden" name="action" value="connected" />
    <input type="hidden" name="controller" value="utilisateur" />
    <p>
      <label for="login_id">Login</label> :
      <input type="text" placeholder="Ex : jdupont" name="login" id="login_id" required autofocus/>
    </p>
    <p>
      <label for="mdp_id">Mot de passe</label> :
      <input type="password" name="mdp" id="mdp_id" required/>
    </p>
    <p>
      <input type="submit" value="Se connecter" />
    </p>
  </fieldset>
</form>
```

Et l’action **connect** qui l’affiche (avec un message d’erreur optionnel) :

```
public static function connect($erreur = NULL) {
    $view = 'connect';
    $pagetitle = 'Connexion';
    require File::build_path(array("view", "view.php"));
}
```

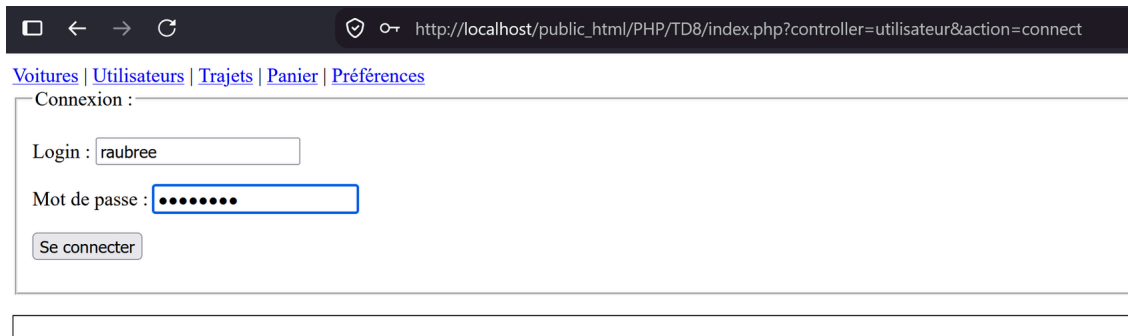
2. La session est déjà démarrée au début de “**index.php**” depuis le **TD7**.

3. Fonction **checkPassword** dans **ModelUtilisateur**, elle renvoie **true** s’il y a exactement un couple login / mot de passe chiffré qui correspond :

```
public static function checkPassword($login, $mot_de_passe_chiffre) {
    $sql = "SELECT * FROM utilisateur WHERE login=:login AND mdp=:mdp";
    $req_prep = Model::$pdo->prepare($sql);
    $values = array(
        "login" => $login,
        "mdp" => $mot_de_passe_chiffre,
    );
    $req_prep->execute($values);
    $tab = $req_prep->fetchAll();
    return (count($tab) == 1);
}
```

Action **connected** : si le couple est valide, on met le login en session et on affiche le détail de l'utilisateur :

```
public static function connected() {
    $login = $_GET['login'];
    $mdp_chiffre = Security::chiffrer($_GET['mdp']);
    if (!ModelUtilisateur::checkPassword($login, $mdp_chiffre)) {
        self::connect("login ou mot de passe incorrect.");
    } else {
        $u = ModelUtilisateur::select($login);
        $_SESSION['login'] = $login;
        $tab_t = ModelUtilisateur::findTrajets($login);
        $view = 'detail';
        $pagetitle = 'Détail de l\'utilisateur';
        require File::build_path(array("view", "view.php"));
    }
}
```



Voitures | Utilisateurs | Trajets | Panier | Préférences

Connexion :

Login : raubree

Mot de passe :

Se connecter



Voitures | Utilisateurs | Trajets | Panier | Préférences

Utilisateur raubree : Romain Aubrée

Trajets en tant que passager :

Aucun trajet.

[Modifier cet utilisateur](#)

[Retour à la liste](#)

Exercice 6

Action **deconnect** qui détruit la session puis renvoie sur la page d'accueil :

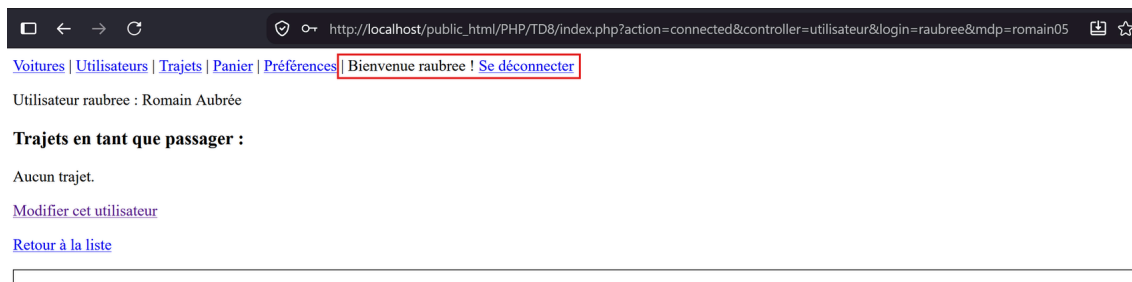
```
public static function deconnect() {  
    session_unset();  
    session_destroy();  
    setcookie(session_name(), '', time() - 1);  
    header('Location: index.php');  
    exit();  
}
```

Exercice 7

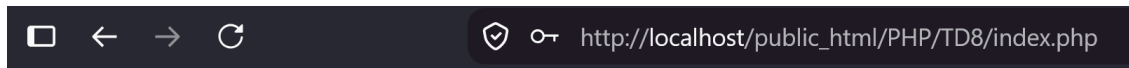
1. Dans le header de “**view.php**”, on affiche un lien de connexion si l'utilisateur n'est pas en session, sinon un message de bienvenue et un lien de déconnexion :

```
<nav>  
    <a href="index.php?action=readAll">Voitures</a> |  
    <a href="index.php?action=readAll&controller=utilisateur">Utilisateurs</a> |  
    <a href="index.php?action=readAll&controller=trajet">Trajets</a> |  
    <a href="index.php?controller=panier">Panier<?php  
    if (!empty($_SESSION['panier'])) {  
        echo " (" . $_SESSION['prix_panier'] . " €)";  
    }  
    ?></a> |  
    <a href="preference.html">Préférences</a> |  
<?php  
if (isset($_SESSION['login'])) {  
    echo '    Bienvenue ' . htmlspecialchars($_SESSION['login']) . ' ! <a href="index.php?controller=utili  
> } else {  
    echo '    <a href="index.php?controller=utilisateur&action=connect">Se connecter</a>';  
> }  
> }  
> }  
> }  
</nav>
```

2. Test avec un bon login / mot de passe :



Après la déconnexion :



Liste des voitures :

Voiture d'immatriculation [&immat=h](#).

Voiture d'immatriculation [00-000-00](#).

Voiture d'immatriculation [AA-000-AA](#).

Voiture d'immatriculation [AB-123-CD](#).

Voiture d'immatriculation [CI-789-TR](#).

Voiture d'immatriculation [FI-456-AT](#).

Voiture d'immatriculation [XX-999-YY](#).

[Ajouter une voiture](#)

Avec un mauvais mot de passe :



[Voitures](#) | [Utilisateurs](#) | [Trajets](#) | [Panier](#) | [Préférences](#) | [Se connecter](#)

Erreur : login ou mot de passe incorrect.

Connexion :

Login :

Mot de passe :

Exercice 8

Création de “lib/Session.php” avec la fonction `is_user()`, inclus dans “index.php” :

```
<?php
class Session {

    public static function is_user($login) {
        return (!empty($_SESSION['login']) && ($_SESSION['login'] == $login));
    }
}
?>
```

```
require_once __DIR__ . $DS . "lib" . $DS . "File.php";
require_once File::build_path(array("lib", "Session.php"));
```

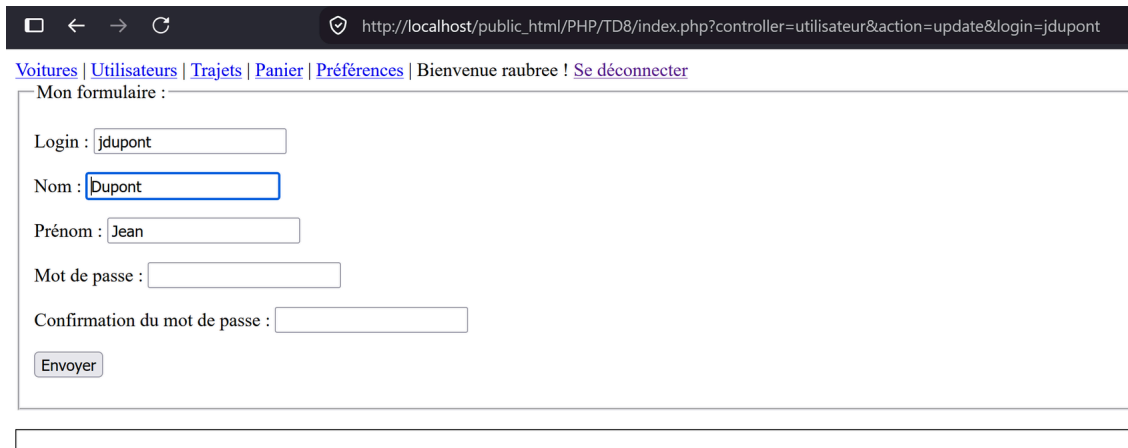
La vue de détail n’affiche les liens de modification et de suppression que si c’est l’utilisateur connecté :

```
<?php
if (Session::is_user($u->getLogin())) {
    echo '<p><a href="index.php?controller=utilisateur&action=update&login=' . $uLoginURL . '">Modifier cet utilisat
    echo '<p><a href="index.php?controller=utilisateur&action=delete&login=' . $uLoginURL . '">Supprimer cet utilisat
}
?>
```



Exercice 9

1. Supprimer le lien ne suffit pas : en tapant l'action **update** dans l'URL, on arrive quand même sur le formulaire d'un autre utilisateur :



Voitures | Utilisateurs | Trajets | Panier | Préférences | Bienvenue raubree ! [Se déconnecter](#)

Mon formulaire :

Login :

Nom :

Prénom :

Mot de passe :

Confirmation du mot de passe :

2. Dans l'action **update**, si le login ne correspond pas à celui de la session, on renvoie vers la page de connexion :

```
public static function update() {
    $login = $_GET['login'];
    if (!Session::is_user($login)) {
        self::connect("vous devez être connecté avec ce compte pour le modifier.");
    } else {
        $u = ModelUtilisateur::select($login);
        if ($u === false) {
            self::error();
        } else {
            $login = $u->getLogin();
            $nom = $u->getNom();
            $prenom = $u->getPrenom();
            $etat_login = "readonly";
            $action_form = "updated";
            $view = 'update';
            $pagetitle = 'Modifier un utilisateur';
            require File::build_path(array("view", "view.php"));
        }
    }
}
```

Voitures | Utilisateurs | Trajets | Panier | Préférences | Bienvenue raubree ! [Se déconnecter](#)

Erreur : vous devez être connecté avec ce compte pour le modifier.

Connexion :

Login :

Mot de passe :

Exercice 10

1. Mais on peut toujours appeler directement l'action **updated** (le formulaire est en **GET**, donc il suffit d'écrire les paramètres dans l'URL) :

Voitures | Utilisateurs | Trajets | Panier | Préférences | Bienvenue raubree ! [Se déconnecter](#)

L'utilisateur jdupont a bien été mis à jour !

Liste des utilisateurs :

Utilisateur [bidon](#)

Utilisateur [bidon2](#)

Utilisateur [jdupont](#)

Utilisateur [mbardet](#)

Utilisateur [pmartin](#)

Utilisateur [raubree \(supprimer\)](#)

[Ajouter un utilisateur](#)

Voitures | Utilisateurs | Trajets | Panier | Préférences | Bienvenue raubree ! [Se déconnecter](#)

Utilisateur jdupont : Jean Hacked

Trajets en tant que passager :

Aucun trajet.

[Retour à la liste](#)

2. On fait la même vérification dans **updated**, c'est la plus importante car c'est elle qui fait vraiment la mise à jour :

```
public static function updated() {
    $login = $_GET['login'];
    if (!Session::is_user($login)) {
        self::connect("vous devez être connecté avec ce compte pour le modifier.");
    } else {
        if ($_GET['mdp'] != $_GET['mdp2']) {
```

http://localhost/public_html/PHP/TD8/index.php?controller=utilisateur&action=updated&login=jdupont&nom=Hacked2&prenom=

[Voitures](#) | [Utilisateurs](#) | [Trajets](#) | [Panier](#) | [Préférences](#) | Bienvenue raubree ! [Se déconnecter](#)

Erreur : vous devez être connecté avec ce compte pour le modifier.

Connexion :

Login :

Mot de passe :

3. Pareil pour l'action **delete** :

```
public static function delete() {
    $login = $_GET['login'];
    if (!Session::is_user($login)) {
        self::connect("vous devez être connecté avec ce compte pour le supprimer.");
    } else {
        ModelUtilisateur::delete($login);
        $tab_u = ModelUtilisateur::selectAll();
        $view = 'deleted';
        $pagetitle = 'Utilisateur supprimé';
        require File::build_path(array("view", "view.php"));
    }
}
```

http://localhost/public_html/PHP/TD8/index.php?controller=utilisateur&action=delete&login=jdupont

[Voitures](#) | [Utilisateurs](#) | [Trajets](#) | [Panier](#) | [Préférences](#) | Bienvenue raubree ! [Se déconnecter](#)

Erreur : vous devez être connecté avec ce compte pour le supprimer.

Connexion :

Login :

Mot de passe :

Exercice 11

1. Ajout d'un champ **admin** de type **boolean** dans la table **utilisateur**, et on met "raubree" administrateur :

Options supplémentaires

			login	nom	prenom	mdp	admin	
☐	 Éditer	 Copier	 Supprimer	bidon	Bidon	Test	5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a...	0
☐	 Éditer	 Copier	 Supprimer	bidon2	Bidon	Deux	449e014b6fecfd69d2cdd03fb57703cbb4c46f42efbcc2d8c...	0
☐	 Éditer	 Copier	 Supprimer	jdupont	Dupont	Jean	3718113ef8628152bb68d9f7e380863d2e2ef4960eebec45cf...	0
☐	 Éditer	 Copier	 Supprimer	mbardet	Bardet	Marion	NULL	0
☐	 Éditer	 Copier	 Supprimer	pmartin	Martin	Paul	NULL	0
☐	 Éditer	 Copier	 Supprimer	raubree	Aubrée	Romain	17e873217a7d38c155c6b7fb8acd2392881e1524ce292932c6...	1

2. L'action **connected** enregistre dans la session si l'utilisateur est admin :

```
public static function connected() {
    $login = $_GET['login'];
    $mdp_chiffre = Security::chiffre($_GET['mdp']);
    if (!ModelUtilisateur::checkPassword($login, $mdp_chiffre)) {
        self::connect("login ou mot de passe incorrect.");
    } else {
        $u = ModelUtilisateur::select($login);
        $_SESSION['login'] = $login;
        $_SESSION['admin'] = ($u->getAdmin() == 1);
        $tab_t = ModelUtilisateur::findTrajets($login);
        $view = 'detail';
        $pagetitle = 'Détail de l\'utilisateur';
        require File::build_path(array("view", "view.php"));
    }
}
```

3. Fonction **is_admin()** dans "Session.php", et un admin a les droits sur tous les utilisateurs (dans **update**, **updated** et **delete**) :

```
public static function is_admin() {
    return (!empty($_SESSION['admin']) && $_SESSION['admin']);
}
```

```

public static function update() {
    $login = $_GET['login'];
    if (!Session::is_user($login) && !Session::is_admin()) {
        self::connect("vous devez être connecté avec ce compte pour le modifier.");
    } else {
        $u = ModelUtilisateur::select($login);
    }
}

```

Connecté en admin, on peut modifier “jdupont” :

Exercice 12

1. Case à cocher “Administrateur ?” dans “**update.php**”, affichée seulement pour un admin :

```

<?php if (Session::is_admin()) { ?>
    <p>
        <input type="checkbox" name="admin" id="admin_id" <?php echo $admin_checked; ?>/>
        <label for="admin_id">Administrateur ?</label>
    </p>
<?php } ?>

```

```

$login = $u->getLogin();
$nom = $u->getNom();
$prenom = $u->getPrenom();
$admin_checked = ($u->getAdmin() == 1) ? "checked" : "";

```

2. Dans **updated**, seul un admin peut changer le champ **admin** (une case décochée n'est pas envoyée, d'où le test) :

```
if (Session::is_admin()) {  
    $data["admin"] = !isset($_GET['admin']) ? 0 : 1;  
}  
ModelUtilisateur::update($data);
```

On passe “**jdupont**” administrateur :

http://localhost/public_html/PHP/TD8/index.php?controller=utilisateur&action=update&login=jdupont

[Voitures](#) | [Utilisateurs](#) | [Trajets](#) | [Panier](#) | [Préférences](#) | Bienvenue raubree ! [Se déconnecter](#)

Mon formulaire :

Login :

Nom :

Prénom :

Mot de passe :

Confirmation du mot de passe :

☒ Administrateur ?

	login	nom	prenom	mdp	admin
☐ Éditer Copier Supprimer	bidon	Bidon	Test	5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a...	0
☐ Éditer Copier Supprimer	bidon2	Bidon	Deux	449e014b6fecfd69d2cdd03fb57703cbb4c46f42efbcc2d8c...	0
☐ Éditer Copier Supprimer	jdupont	Dupont	Jean	35be6bce90652ed79979282af728a86478faf38b7a64a86a78...	1
☐ Éditer Copier Supprimer	mbardet	Bardet	Marion	NULL	0
☐ Éditer Copier Supprimer	pmartin	Martin	Paul	NULL	0
☐ Éditer Copier Supprimer	raubree	Aubrée	Romain	17e873217a7d38c155c6b7fb8acd2392881e1524ce292932c6...	1

Exercice 13

La table **utilisateur** n'avait pas de champ email, on ajoute une colonne **email VARCHAR(64)** :

Structure de table		Vue relationnelle										
	#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action		
☐	1	login	varchar(32)	utf8_general_ci		Non	Aucun(e)			Modifier	Supprimer	Plus ▼
☐	2	nom	varchar(32)	utf8_general_ci		Oui	NULL			Modifier	Supprimer	Plus ▼
☐	3	prenom	varchar(32)	utf8_general_ci		Oui	NULL			Modifier	Supprimer	Plus ▼
☐	4	email	varchar(64)	utf8_general_ci		Oui	NULL			Modifier	Supprimer	Plus ▼
☐	5	mdp	varchar(64)	utf8_general_ci		Oui	NULL			Modifier	Supprimer	Plus ▼
☐	6	admin	tinyint(1)			Non	0			Modifier	Supprimer	Plus ▼

1. Dans le formulaire, on utilise **type="email"** du HTML5 :

```
<label for="email_id">Email</label> :  
<input type="email" placeholder="Ex : jdupont@yopmail.com" name="email" id="email_id" value="<?php echo $uEmail; ?
```

http://localhost/public_html/PHP/TD8/index.php?controller=utilisateur&action=create

[Voitures](#) | [Utilisateurs](#) | [Trajets](#) | [Panier](#) | [Préférences](#) | Bienvenue raubree ! [Se déconnecter](#)

Mon formulaire :

Login :

Nom :

Prénom :

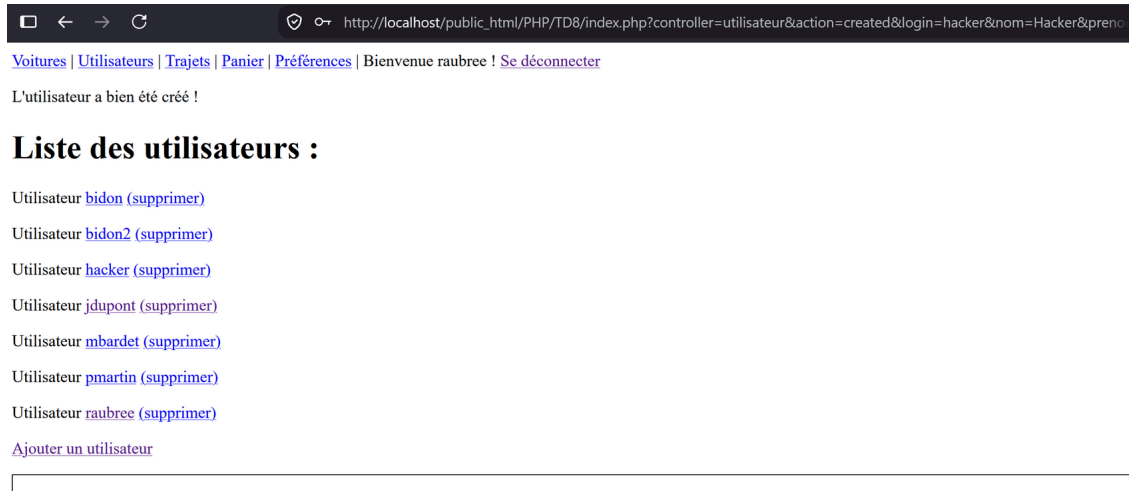
Email :

Mot de passe : Veuillez saisir une adresse e-mail valide.

Confirmation du mot de passe :

☐ Administrateur ?

2. Mais ce contrôle se fait dans le navigateur, on peut facilement le contourner. Comme le formulaire est en **GET**, on écrit directement l'URL avec une adresse invalide :

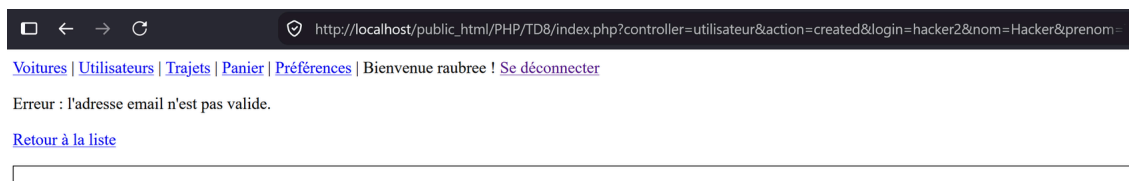


L'utilisateur a été créé avec l'adresse "**pasunmail**". Les contrôles côté client ne sont donc qu'un confort, pas une sécurité.

3. Vérification côté serveur dans l'action **created** avec **filter_var()** et **FILTER_VALIDATE_EMAIL** :

```
public static function created() {
    if ($_GET['mdp'] != $_GET['mdp2']) {
        self::error("les deux mots de passe ne correspondent pas.");
    } else if (!filter_var($_GET['email'], FILTER_VALIDATE_EMAIL)) {
        self::error("l'adresse email n'est pas valide.");
    } else {
        $data = array(
            "login" => $_GET['login'],

```



Exercice 14

1. Ajout d'un champ **nonce** **VARCHAR(32)** dans la table **utilisateur** :

Structure de table											
	#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action	
☐	1	login	varchar(32)	utf8_general_ci		Non	Aucun(e)			 Modifier	 Supprimer Plus ▾
☐	2	nom	varchar(32)	utf8_general_ci		Oui	NULL			 Modifier	 Supprimer Plus ▾
☐	3	prenom	varchar(32)	utf8_general_ci		Oui	NULL			 Modifier	 Supprimer Plus ▾
☐	4	email	varchar(64)	utf8_general_ci		Oui	NULL			 Modifier	 Supprimer Plus ▾
☐	5	mdp	varchar(64)	utf8_general_ci		Oui	NULL			 Modifier	 Supprimer Plus ▾
☐	6	admin	tinyint(1)			Non	0			 Modifier	 Supprimer Plus ▾
☐	7	nonce	varchar(32)	utf8_general_ci		Oui	NULL			 Modifier	 Supprimer Plus ▾

2. L'action **connected** n'accepte la connexion que si le **nonce** est **NULL** :

```
public static function connected() {
    $login = $_GET['login'];
    $mdp_chiffre = Security::chiffrer($_GET['mdp']);
    if (!ModelUtilisateur::checkPassword($login, $mdp_chiffre)) {
        self::connect("login ou mot de passe incorrect.");
    } else {
        $u = ModelUtilisateur::select($login);
        if (!is_null($u->getNonce())) {
            self::connect("vous devez d'abord valider votre adresse email.");
        } else {
            $_SESSION['login'] = $login;
            $_SESSION['admin'] = ($u->getAdmin() == 1);
            $tab_t = ModelUtilisateur::findTrajets($login);
            $view = 'detail';
            $pagetitle = 'Détail de l\'utilisateur';
            require File::build_path(array("view", "view.php"));
        }
    }
}
```

3. Action **validate** qui passe le **nonce** à **NULL** si le login et le nonce de l'URL correspondent :

```
public static function validate() {
    $login = $_GET['login'];
    $nonce = $_GET['nonce'];
    $u = ModelUtilisateur::select($login);
    if ($u !== false && !is_null($u->getNonce()) && $u->getNonce() == $nonce) {
        ModelUtilisateur::update(array("login" => $login, "nonce" => NULL));
        $view = 'validated';
        $pagetitle = 'Email validé';
        require File::build_path(array("view", "view.php"));
    } else {
        self::error("le lien de validation n'est pas valide.");
    }
}
```

4. Fonction **generateRandomHex()** dans **Security**, et dans **created** on génère le nonce :

```
public static function generateRandomHex() {
    // Generate a 32 digits hexadecimal number
    $numbytes = 16; // Because 32 digits hexadecimal = 16 bytes
    $bytes = openssl_random_pseudo_bytes($numbytes);
    $hex = bin2hex($bytes);
    return $hex;
}
```

```

public static function created() {
    if ($_GET['mdp'] != $_GET['mdp2']) {
        self::error("les deux mots de passe ne correspondent pas.");
    } else if (!filter_var($_GET['email'], FILTER_VALIDATE_EMAIL)) {
        self::error("l'adresse email n'est pas valide.");
    } else {
        $nonce = Security::generateRandomHex();
        $data = array(
            "login" => $_GET['login'],
            "nom" => $_GET['nom'],
            "prenom" => $_GET['prenom'],
            "email" => $_GET['email'],
            "mdp" => Security::chiffrer($_GET['mdp']),
            "nonce" => $nonce,
        );
        if (ModelUtilisateur::save($data) === false) {
            self::error("ce login est déjà utilisé.");
        } else {
            self::envoyerMail($_GET['login'], $_GET['email'], $nonce);
            $tab_u = ModelUtilisateur::selectAll();
            $view = 'created';
            $pagetitle = 'Utilisateur créé';
            require File::build_path(array("view", "view.php"));
        }
    }
}

```

5. Le mail est écrit en **HTML** dans **\$mail** avec un lien vers l'action **validate** (login et nonce en query string), puis envoyé avec **mail()** :

```

private static function envoyerMail($login, $email, $nonce) {
    $lien = "http://" . $_SERVER['HTTP_HOST'] . dirname($_SERVER['PHP_SELF'])
        . "/index.php?controller=utilisateur&action=validate&login=" . rawurlencode($login) . "&nonce=" . $nonce;

    $mail = "<html><body>"
        . "<p>Bonjour " . htmlspecialchars($login) . ",</p>"
        . "<p>Merci pour votre inscription sur le site de covoiturage.</p>"
        . "<p>Pour valider votre adresse email, cliquez sur ce lien :<br>"
        . "<a href=\"" . $lien . "\"> " . $lien . "</a></p>"
        . "</body></html>";

    $headers = "MIME-Version: 1.0\r\n";
    $headers .= "Content-type: text/html; charset=UTF-8\r\n";
    $headers .= "From: covoiturage@localhost\r\n";
    mail($email, "Validation de votre inscription", $mail, $headers);
}

```

Pour ne pas envoyer de vrais mails depuis ma machine, j'ai configuré **XAMPP** pour que **mail()** écrive les mails dans des fichiers ("**mailtodisk**", dans le dossier "**C:\xampp\mailoutput**") en modifiant "**php.ini**" :

```

; https://php.net/sendmail-path
sendmail_path = "\"C:\xampp\mailtodisk\mailtodisk.exe\"

```

Test en créant un utilisateur avec une adresse **yopmail** :

Voitures | Utilisateurs | Trajets | Panier | Préférences | Bienvenue raubree ! [Se déconnecter](#)

Mon formulaire :

Login :

Nom :

Prénom :

Email :

Mot de passe :

Confirmation du mot de passe :

☐ Administrateur ?

Voitures | Utilisateurs | Trajets | Panier | Préférences | Bienvenue raubree ! [Se déconnecter](#)

L'utilisateur a bien été créé ! Un email de validation vient de lui être envoyé.

Liste des utilisateurs :

Utilisateur [bidon](#) ([supprimer](#))

Utilisateur [bidon2](#) ([supprimer](#))

Utilisateur [hacker](#) ([supprimer](#))

Utilisateur [jdupont](#) ([supprimer](#))

Utilisateur [lmartin](#) ([supprimer](#))

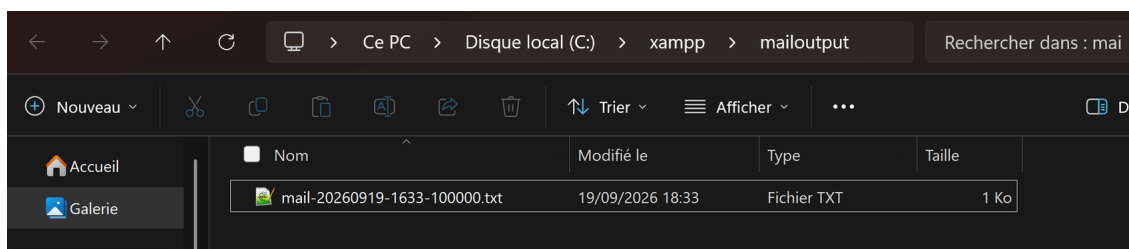
Utilisateur [mbardet](#) ([supprimer](#))

Utilisateur [pmartin](#) ([supprimer](#))

Utilisateur [raubree](#) ([supprimer](#))

[Ajouter un utilisateur](#)

Le mail arrive dans “mailoutput” :



```
To: lmartin@yopmail.com
Subject: Validation de votre inscription
MIME-Version: 1.0
Content-type: text/html; charset=UTF-8
From: covoiturage@localhost

<html><body><p>Bonjour lmartin,</p><p>Merci pour votre inscription sur le site de covoiturage.</p><p>Pour valider votre adresse email, cliquez sur le lien ci-dessous :</p><p><a href="http://localhost/public_html/PHP/TD8/index.php?controller=utilisateur&action=validate&login=lmartin&nonce=5d00e274773d52e9400d967b6c40c09d">http://localhost/public_html/PHP/TD8/index.php?controller=utilisateur&action=validate&login=lmartin&nonce=5d00e274773d52e9400d967b6c40c09d</a></p></body></html>
```

Si **lmartin** essaie de se connecter avant d'avoir cliqué sur le lien :

The screenshot shows a web browser at the URL `http://localhost/public_html/PHP/TD8/index.php?action=connected&controller=utilisateur&login=lmartin&mdp=lucas05`. The page has a navigation bar with links: [Voitures](#) | [Utilisateurs](#) | [Trajets](#) | [Panier](#) | [Préférences](#) | [Se connecter](#). Below the navigation bar, a red error message states: "Erreur : vous devez d'abord valider votre adresse email." Below the error message, there is a login form titled "Connexion :". The form contains a "Login :" label with a text input field containing "Ex : jdupont", a "Mot de passe :" label with a password input field, and a "Se connecter" button.

On clique sur le lien du mail :

The screenshot shows a web browser at the URL `http://localhost/public_html/PHP/TD8/index.php?controller=utilisateur&action=validate&login=lmartin&nonce=41be15575db3f1`. The page has a navigation bar with links: [Voitures](#) | [Utilisateurs](#) | [Trajets](#) | [Panier](#) | [Préférences](#) | [Se connecter](#). Below the navigation bar, a message states: "Votre adresse email a bien été validée, vous pouvez maintenant vous connecter." Below the message, there is a "Se connecter" button.

Et maintenant la connexion marche :

The screenshot shows a web browser at the URL `http://localhost/public_html/PHP/TD8/index.php?action=connected&controller=utilisateur&login=lmartin&mdp=lucas05`. The page has a navigation bar with links: [Voitures](#) | [Utilisateurs](#) | [Trajets](#) | [Panier](#) | [Préférences](#) | [Bienvenue lmartin !](#) | [Se déconnecter](#). Below the navigation bar, the text "Utilisateur lmartin : Lucas Martin" is displayed. Below the text, the heading "Trajets en tant que passager :" is shown. Below the heading, the text "Aucun trajet." is displayed. Below the text, there are three links: [Modifier cet utilisateur](#), [Supprimer cet utilisateur](#), and [Retour à la liste](#).

Options supplémentaires									
←T→		login	nom	prenom	email	mdp		admin	nonce
☐ Éditer  Copier  Supprimer	bidon	Bidon	Test	NULL	5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a...			0	NULL
☐ Éditer  Copier  Supprimer	bidon2	Bidon	Deux	NULL	449e014b6fecfd69d2cdd03fb57703cbb4c46f42efbcc2d8c...			0	NULL
☐ Éditer  Copier  Supprimer	hacker	Hacker	Test	pasunmail	51ea8ddc11c215f2da7ebbe3285914c9fcefd454f204d95e1c...			0	NULL
☐ Éditer  Copier  Supprimer	jdupont	Dupont	Jean	NULL	35be6bce90652ed79979282af728a86478faf38b7a64a86a78...			1	NULL
☐ Éditer  Copier  Supprimer	lmartin	Martin	Lucas	lmartin@yopmail.com	371b350ceceebecb48563ecf73dd8d4f08451fdae085b42202...			0	NULL
☐ Éditer  Copier  Supprimer	mbardet	Bardet	Marion	NULL	NULL			0	NULL
 Console de requêtes SQL  Supprimer	pmartin	Martin	Paul	NULL	NULL			0	NULL

Exercice 15

Pour l'instant le mot de passe passe en clair dans l'URL :



1. Fonction **myGet()** dans le routeur, elle renvoie la variable qu'elle soit en **GET** ou en **POST** :

```
function myGet($nomvar) {
    if (isset($_GET[$nomvar])) {
        return $_GET[$nomvar];
    } else if (isset($_POST[$nomvar])) {
        return $_POST[$nomvar];
    } else {
        return NULL;
    }
}
```


2. On remplace tous les `$_GET` du routeur et des contrôleurs par `myGet()` (avec Ctrl+H), et les `isset($_GET['...'])` par `!is_null(myGet('...'))` :

```
if (!is_null(myGet('action'))) {
    $action = myGet('action');
} else {
    $action = 'readAll';
}

$controller_default = 'voiture';
if (isset($_COOKIE['preference'])) {
    $controller_default = $_COOKIE['preference'];
}

if (!is_null(myGet('controller'))) {
    $controller = myGet('controller');
} else {
    $controller = $controller_default;
}

$controller_class = 'Controller' . ucfirst($controller);

if (class_exists($controller_class)) {
    if (in_array($action, get_class_methods($controller_class))) {
        $controller_class::$action();
    } else {
        ControllerVoiture::error();
    }
} else {
    ControllerVoiture::error();
}
?>
```

```
public static function connected() {
    $login = myGet('login');
    $mdp_chiffre = Security::chiffre(myGet('mdp'));
    if (!ModelUtilisateur::checkPassword($login, $mdp_chiffre)) {
```

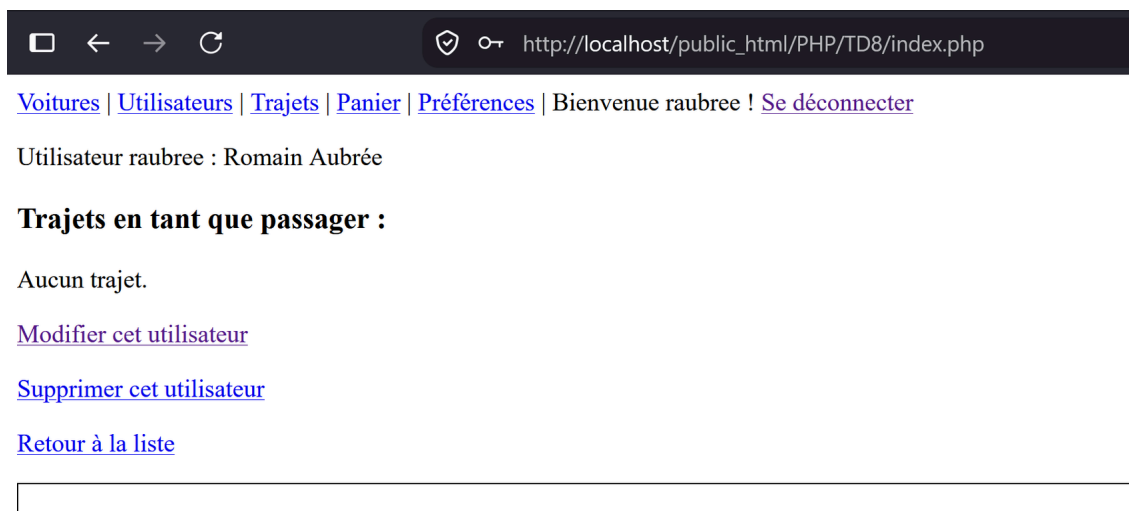
3. Les formulaires sont en **POST** si **Conf::getDebug()** est à **false** (site en production), sinon en **GET** :

```
<?php
$uLogin = htmlspecialchars($login);
$uNom = htmlspecialchars($nom);
$uPrenom = htmlspecialchars($prenom);
$uEmail = htmlspecialchars($email);
$methode = Conf::getDebug() ? "get" : "post";
?>
<form method="<?php echo $methode; ?>" action="index.php">
```

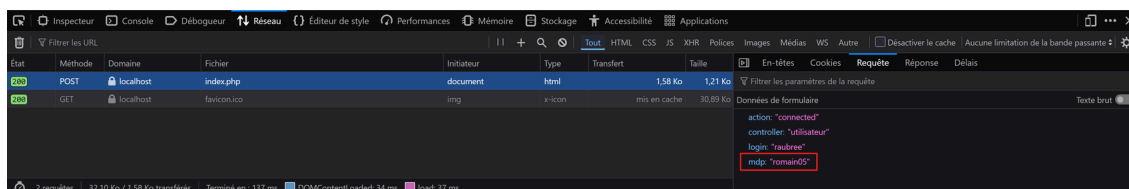
On passe **\$debug** à **false** dans “**Conf.php**” :

```
// la variable debug est un boolea
static private $debug = False;
```

Les paramètres ne sont plus dans l’URL :



Par contre, le mot de passe est toujours visible en clair dans l’onglet **Réseau** des outils de développement, car il est envoyé en clair dans le corps de la requête :



Le hachage protège les mots de passe si quelqu'un arrive à lire la **BDD**, mais pour protéger la communication entre le navigateur et le serveur, il faudrait passer le site en **HTTPS**.

Conclusion

Ce **TD** nous a appris à gérer l'authentification sur un site : mots de passe hachés et salés, connexion avec les sessions, vérification des droits côté serveur (utilisateur / admin), validation de l'adresse email avec un nonce, et passage des formulaires en **POST**. La règle principale : ne jamais faire confiance au client.